

InsightForge

Autonomous Data Scientist Agent

A Multi-Agent AI Platform for End-to-End Data Science Automation

Developer: Tanveer Singh Bedi
Type: Full-Stack AI / Agentic System
Year: 2025

Table of Contents

- 1. Abstract**
- 2. Introduction**
 - 2.1 Background & Motivation
 - 2.2 Problem Statement
 - 2.3 Objectives
 - 2.4 Scope of the Project
- 3. System Architecture**
 - 3.1 High-Level Three-Tier Overview
 - 3.2 LangGraph Agent Graph
 - 3.3 Shared State Schema
- 4. Technology Stack**
 - 4.1 FastAPI
 - 4.2 LangGraph
 - 4.3 scikit-learn
 - 4.4 XGBoost
 - 4.5 React + Vite
- 5. Agent Pipeline — Detailed Walkthrough**
 - 5.1–5.9 Agents 1–9 (Plan → Report)
- 6. Frontend Design**
 - 6.1 Application Flow (3-Phase Wizard)
 - 6.2 Results Dashboard Tabs
 - 6.3 State Management
- 7. Testing & Results**
 - 7.1 Test Datasets Overview
 - 7.2 Telco Customer Churn — Detailed Case Study
 - 7.3 Bugs Identified and Fixed
- 8. Challenges & Solutions**
- 9. Future Scope**
- 10. Conclusion**
- 11. References**

1. Abstract

InsightForge is a full-stack, multi-agent AI platform that automates the complete data science pipeline from raw CSV upload to final model report — requiring zero code from the end user. Built on a stateful nine-node LangGraph orchestration graph and a FastAPI backend, the system sequentially executes data cleaning, exploratory data analysis, feature engineering, model selection, hyperparameter tuning, evaluation, SHAP-based explainability, and natural-language report generation.

- Problem solved: eliminates the 80 % of DS project time spent on boilerplate setup, encoding pipelines, and reproducibility concerns.
- Architecture: nine specialised LangGraph agents share a TypedDict state; every artefact (DataFrames, trained models, evaluation metrics) flows through a single graph without manual handoff code.
- Stack: Python 3.11, FastAPI 0.110, LangGraph 0.1, scikit-learn 1.4, XGBoost 2.0, React 18 + Vite + Tailwind CSS, Groq LLM API.
- Key result: on the Telco Customer Churn benchmark (7,043 rows, 20 features, 2.77 × class imbalance), InsightForge selected and tuned XGBoost to an F1 score of 0.7973 in 1.8 seconds of training time.
- Impact: non-technical domain experts can obtain a fully reproducible ML pipeline and professional PDF-ready report in under 60 seconds.

KEY TAKEAWAY *InsightForge demonstrates that a multi-agent LangGraph architecture can replace hundreds of lines of bespoke glue code while simultaneously improving reproducibility, auditability, and accessibility.*

2. Introduction

2.1 Background & Motivation

- Every data science project follows an almost identical sequence: load data, clean, encode, engineer features, train, tune, evaluate, and document — yet practitioners rebuild this scaffolding from scratch for each project.
- Non-technical domain experts — clinicians, financial analysts, operations managers — possess the domain knowledge needed to define the problem but lack the Python skills to participate in ML workflows.
- Manual pipelines introduce subtle but costly bugs: label encoders fitted on full data before splitting cause data leakage; inconsistent imputation strategies produce different results across runs.
- The emergence of LLM-orchestrated agentic systems (LangChain, LangGraph, AutoGen) enables a new paradigm: decompose the pipeline into specialised, stateful agents that reason and act autonomously.
- InsightForge capitalises on this paradigm to deliver a zero-code, end-to-end DS platform accessible through a browser.

2.2 Problem Statement

- Every DS project follows the invariant loop: load → clean → encode → engineer → train → tune → evaluate → document, consuming days of engineering time per project.
- Setting up GridSearchCV, stratified splits, SMOTE oversampling, SHAP explainability, and a coherent evaluation report requires deep library knowledge that most domain users do not possess.
- High risk of data leakage: fitting transformers before the train/test split is one of the most common and costly mistakes in production ML.
- Lack of reproducibility: ad-hoc Jupyter notebooks accumulate hidden state and are difficult to re-run on new data.
- Non-technical stakeholders cannot participate in or validate ML workflows without programmer assistance.

2.3 Objectives

1. Automate the full data science pipeline end-to-end with no user code required.
2. Build a stateful multi-agent system using LangGraph with explicit, auditable state transitions.
3. Provide an intuitive, three-phase React frontend accessible to non-technical users.
4. Produce explainable, reproducible results via SHAP values and per-class evaluation metrics.
5. Support both classification and regression tasks across arbitrary tabular CSV datasets.

2.4 Scope of the Project

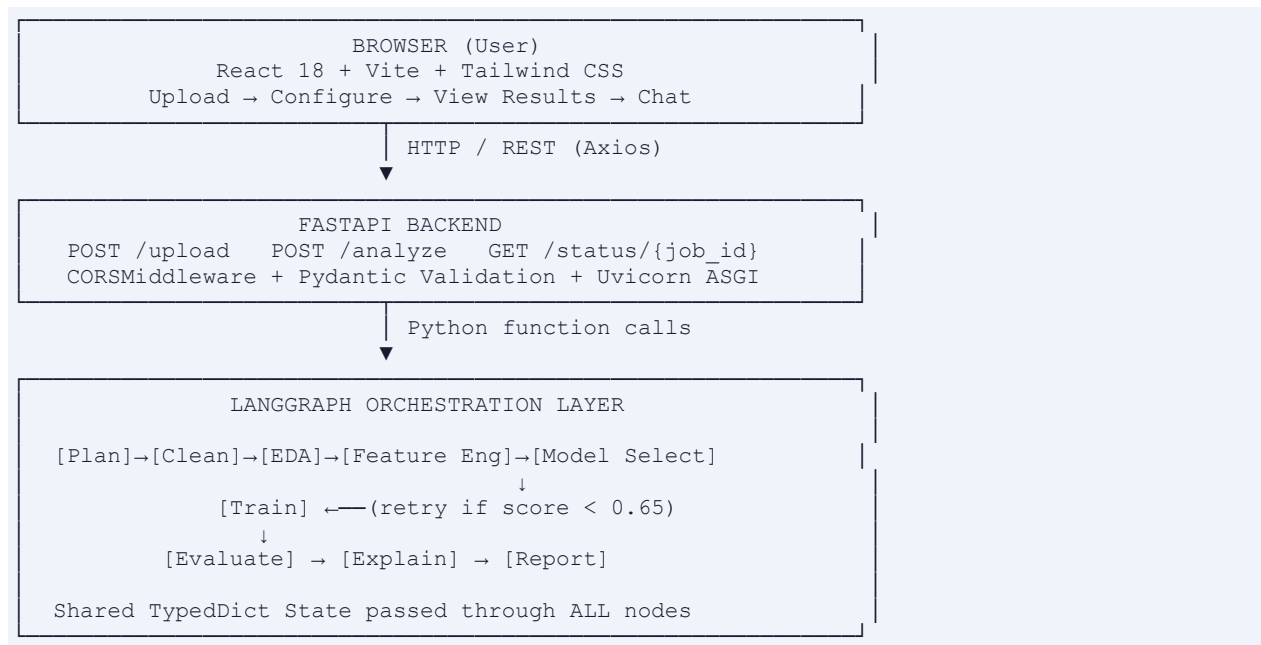
- Input formats accepted: CSV, Excel (.xlsx), Parquet.
- Task types supported: binary classification, multi-class classification, regression.
- Outputs delivered: cleaned dataset, EDA summary statistics, trained and tuned model, SHAP feature importances, confusion matrix, and a downloadable natural-language final report.
- Current limitations: no real-time streaming data, no native NLP/text classification pipeline, no time-series forecasting (planned in future scope).

KEY TAKEAWAY *The scope is intentionally focused on structured tabular data — the most common format in enterprise analytics — to deliver a polished, production-quality experience rather than a broad but shallow tool.*

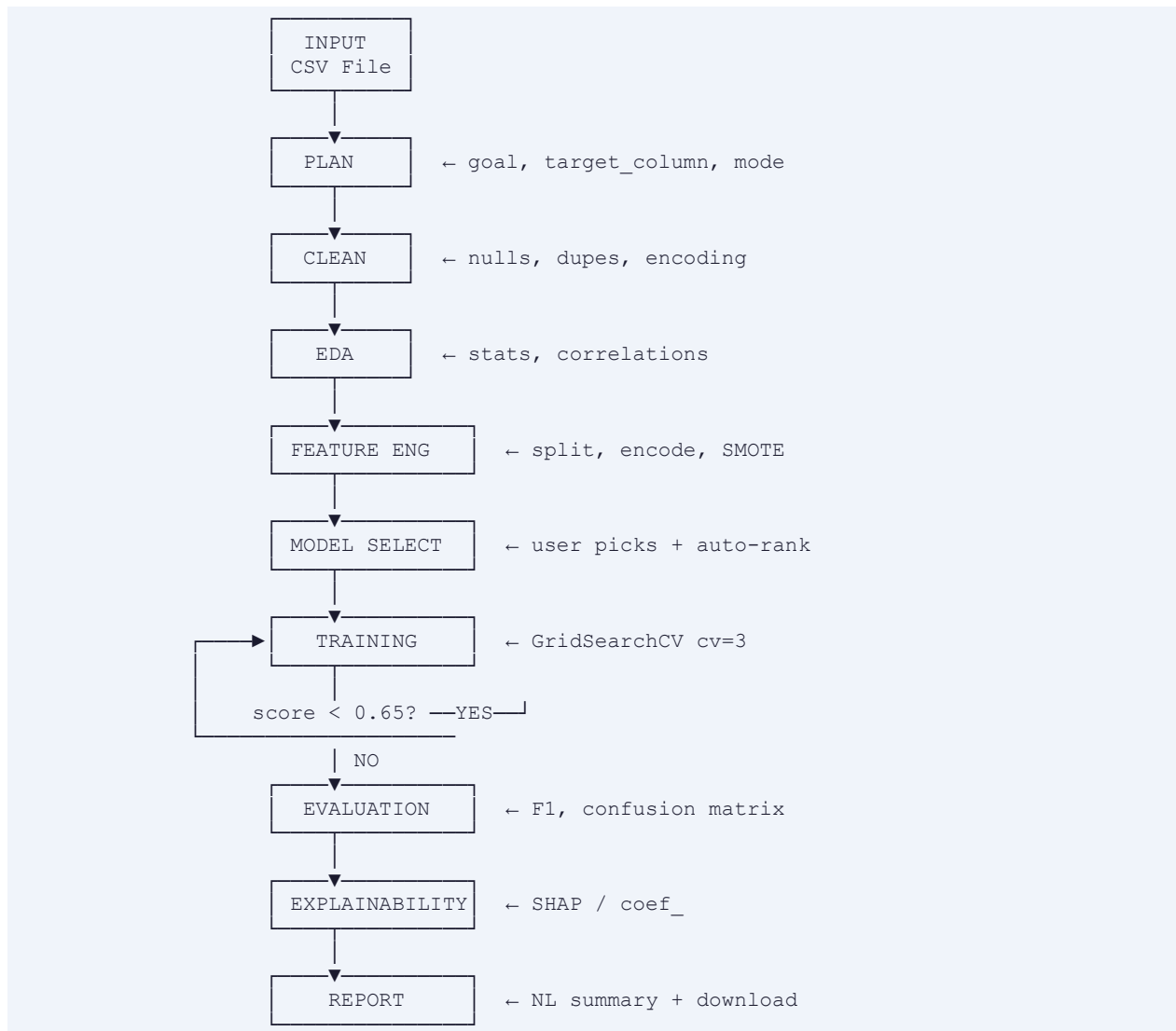
3. System Architecture

3.1 High-Level Three-Tier Overview

InsightForge follows a strict three-tier architecture: a React 18 presentation layer, a FastAPI application layer, and a LangGraph orchestration layer. Each tier has a single responsibility and communicates through well-defined interfaces.



3.2 LangGraph Agent Graph



3.3 Shared State Schema

All nine agents read from and write to a single LangGraph TypedDict state object. The table below documents every key, its type, which agent sets it, and which agents consume it.

State Key	Type	Set By	Used By
df_raw	DataFrame	Upload endpoint	Cleaning Agent
df_clean	DataFrame	Cleaning Agent	EDA, Feature Eng
cleaning_log	list[str]	Cleaning Agent	Report Agent
quality_score	int (0–100)	Cleaning Agent	Frontend

State Key	Type	Set By	Used By
eda_stats	dict	EDA Agent	Feature Eng, Report
task_type	str (CLASS / REG)	EDA Agent	Model Selector
class_imbalance_ratio	float	EDA Agent	Feature Eng
X_train, X_test, y_train, y_test	ndarray	Feature Eng	Training Agent
feature_names	list[str]	Feature Eng	Explainability
feature_report	dict	Feature Eng	Frontend
trained_models	dict	Training Agent	Evaluation, Explainability
best_model_name	str	Evaluation Agent	Explainability, Report
evaluation_report	dict	Evaluation Agent	Report Agent
confusion_matrix	list[list]	Evaluation Agent	Frontend
shap_values / feature_importances	dict	Explainability Agent	Frontend
final_report	str	Report Agent	Frontend download

KEY TAKEAWAY *The shared TypedDict state is the core innovation of InsightForge: it eliminates inter-agent serialisation overhead and makes every intermediate artefact auditable at any point in the pipeline.*

4. Technology Stack

4.1 FastAPI

- Why chosen: async-native ASGI framework with automatic OpenAPI docs, Pydantic validation, and sub-millisecond routing overhead — far superior to Flask for concurrent long-running ML jobs.
- How used: exposes three primary endpoints — POST /upload (multipart CSV ingestion), POST /analyze (triggers LangGraph graph with job_id), GET /status/{job_id} (polling endpoint returning live agent progress).
- CORSMiddleware configured to allow requests from the React dev server at localhost:5173 during development.
- Uvicorn ASGI server with workers=1 to avoid shared-memory conflicts in the LangGraph state dictionary.

4.2 LangGraph

- Why chosen: only framework offering TypedDict shared state, built-in conditional edges (for retry loops), and `graph.compile()` for visualising the execution DAG before deployment.
- How used: nine nodes registered via `graph.add_node()`; edges defined with `graph.add_edge()` and `graph.add_conditional_edges()` for the training retry path (score < 0.65).
- StateGraph compiled with `graph.compile()` produces a Runnable that accepts the initial state dict and streams events to the FastAPI job-status store.

Criterion	Plain Python	LangChain Chains	Celery Tasks	LangGraph
Shared state across all steps	Manual	Only prev step	Manual	TypedDict state
Conditional branching (loops)	Manual	No	Manual	Built-in edges
Visual graph compilation	No	No	No	<code>graph.compile()</code>
Retry logic	Manual	Limited	Manual	Conditional edges
Agent isolation	No	No	Yes	Yes
Development complexity	Low	Medium	High	Medium

4.3 scikit-learn

- Provides Logistic Regression, Random Forest, Gradient Boosting, SVM, Ridge, and SVR implementations.
- GridSearchCV with `cv=3` used for all tunable models; StratifiedKFold used for classification to preserve class proportions across folds.
- Pipeline objects used inside the Training Agent to chain StandardScaler → estimator, ensuring the scaler is fitted only on training folds — preventing data leakage.

4.4 XGBoost

- Gradient-boosted tree library that consistently outperforms scikit-learn ensemble methods on tabular data with moderate size.
- Used via the XGBClassifier and XGBRegressor wrappers; integrates natively with scikit-learn Pipeline and GridSearchCV.
- Best hyperparameters on Telco Churn: {subsample: 0.7, n_estimators: 300, max_depth: 3, learning_rate: 0.05}.
- SHAP TreeExplainer natively supports XGBoost models, producing exact Shapley values without approximation.

4.5 React + Vite

- React 18 with concurrent features enables the three-phase wizard UI without page reloads or route changes.
- Vite provides HMR (Hot Module Replacement) in under 300 ms, dramatically accelerating frontend development iteration.
- Tailwind CSS utility classes used for all styling; no custom CSS files — ensures design consistency and rapid prototyping.
- Recharts library renders the confusion matrix heatmap, SHAP bar chart, and F1 model comparison table with responsive SVG outputs.

Task	Algorithm	Library	Tuned?
Classification	Logistic Regression	scikit-learn	No
Classification	Random Forest	scikit-learn	Yes
Classification	XGBoost	xgboost	Yes
Classification	Gradient Boosting	scikit-learn	Yes
Classification	SVM	scikit-learn	No
Regression	Linear Regression	scikit-learn	No
Regression	Random Forest	scikit-learn	Yes
Regression	XGBoost	xgboost	Yes
Regression	Gradient Boosting	scikit-learn	Yes
Regression	Ridge / SVR	scikit-learn	No

Environment Configuration Variables

Variable	Default	Description
GROQ_API_KEY	Required	API key for Groq LLM service
LLM_MODEL_NAME	llama-3.3-70b-versatile	Groq model identifier
LLM_TEMPERATURE	0.0	0 = deterministic, 1 = creative
FAST_MODE	false	Skip SHAP and heavy visualisations
STORAGE_PATH	./storage	Local path for datasets and models

KEY TAKEAWAY *LangGraph was the decisive technical choice — its TypedDict shared state eliminated ~400 lines of serialisation and state-passing code that would have been required with plain Python or Celery.*

5. Agent Pipeline — Detailed Walkthrough

The table below summarises all nine agents before their individual deep-dives.

#	Agent	Key Input	Key Output	Special Logic
1	Plan Agent	goal text, target_col	plan dict	Validates target column exists
2	Cleaning Agent	df_raw	df_clean, cleaning_log	Drops ID cols (unique_ratio > 0.95)
3	EDA Agent	df_clean	eda_stats, task_type	Infers CLASSIFICATION vs REGRESSION
4	Feature Engineer	df_clean, eda_stats	X_train/test splits	SMOTE if imbalance_ratio > 1.5
5	Model Selector	task_type, user picks	model_list + grids	Priority: XGBoost → Ensemble → Linear
6	Training Agent	X_train, model_list	trained_models	GridSearchCV cv=3; retry if F1 < 0.65
7	Evaluation Agent	trained_models, X_test	evaluation_report	Confusion matrix, per-class F1
8	Explainability	best_model, feature_names	feature_importances	SHAP for trees, abs(coef_) for linear
9	Report Agent	full state	final_report string	NL summary + download

5.1 Agent 1: Plan Agent

Purpose: Parses the user-supplied goal statement and validates that the target column exists in the uploaded dataset.

Inputs:

- goal: free-text description of the analysis objective
- target_column: name of the label column
- df_raw: the raw uploaded DataFrame

Processing Logic:

- Checks target_column in df_raw.columns; raises PlanError if absent
- Infers whether the task is likely classification or regression based on target cardinality (> 20 unique values → REGRESSION hint)
- Constructs a plan dict with keys: goal, target_col, inferred_mode, timestamp
- Logs the validated plan to the shared state for audit trail

Outputs:

- plan dict written to state['plan']
- Passes df_raw unchanged to the next agent

UI Display:

- Displays a green "Plan Validated" badge on the frontend status tracker
- Shows goal text and target column name in the Phase 3 results header

5.2 Agent 2: Cleaning Agent

Purpose: Detects and resolves data quality issues including missing values, duplicate rows, constant columns, and high-cardinality identifier columns.

Inputs:

- `df_raw`: raw DataFrame
- `target_column`: must not be dropped

Processing Logic:

- Drops duplicate rows; logs count to `cleaning_log`
- Imputes numeric nulls with column median; categorical nulls with mode
- Drops columns where `null_rate > 0.6` (configurable threshold)
- Auto-detects and drops ID-like columns where `unique_ratio > 0.95` (e.g., `customerID`)
- Drops constant columns (`std == 0`)
- Computes `quality_score = 100 - (weighted penalties for nulls, dupes, high cardinality)`

Outputs:

- `df_clean`: cleaned DataFrame
- `cleaning_log`: list of human-readable action strings
- `quality_score`: integer 0–100

UI Display:

- Quality score displayed as an animated gauge widget
- Cleaning log rendered as a scrollable action-by-action audit list
- Stats cards show: rows removed, nulls fixed, columns dropped

5.3 Agent 3: EDA Agent

Purpose: Produces a statistical summary of the cleaned dataset and determines the definitive task type for downstream agents.

Inputs:

- `df_clean`
- `target_column`

Processing Logic:

- Calls `df.describe()` and stores result in `eda_stats['summary']`
- Computes Pearson correlation matrix; stores top correlations with target
- Infers `task_type`: if target dtype is object or `nunique() < 15` → CLASSIFICATION; else REGRESSION
- Computes `class_imbalance_ratio = max_class_count / min_class_count` for classification tasks

- Identifies skewed numeric columns (skewness > 1.5) for Feature Eng to log-transform

Outputs:

- eda_stats dict with keys: summary, correlations, skewed_cols, target_distribution
- task_type string: CLASSIFICATION or REGRESSION
- class_imbalance_ratio float

UI Display:

- EDA tab: statistical summary table, top-N feature correlation bar chart
- Target distribution shown as donut chart
- Skewed columns highlighted in preprocessing tab

5.4 Agent 4: Feature Engineer Agent

Purpose: Splits data, encodes categorical features, scales numeric features, and applies SMOTE oversampling when class imbalance is detected.

Inputs:

- df_clean
- eda_stats (for skewed_cols and task_type)
- target_column

Processing Logic:

- Performs train_test_split with test_size=0.2 and stratify=y (classification only)
- Label-encodes binary categoricals; one-hot-encodes multi-class categoricals on X_train, then applies the same transform to X_test (no leakage)
- Applies StandardScaler fitted only on X_train; transforms X_test
- Log-transforms skewed numeric columns identified by EDA Agent
- Applies SMOTE(random_state=42) when class_imbalance_ratio > 1.5 — only on X_train to prevent leakage
- Stores feature_names list (post-encoding column names) for SHAP alignment

Outputs:

- X_train, X_test, y_train, y_test ndarrays
- feature_names list
- feature_report dict (scaler type, SMOTE flag, train/test sizes)

UI Display:

- Preprocessing tab displays: scaler used, SMOTE applied (yes/no), train size, test size
- Feature list with cardinality metadata shown in collapsible accordion

5.5 Agent 5: Model Selector Agent

Purpose: Assembles the list of models to train based on task type and user preferences, along with their hyperparameter grids.

Inputs:

- task_type

- user_model_preferences (from API request body)

Processing Logic:

- Priority ordering: XGBoost → Gradient Boosting → Random Forest → Logistic Regression / Linear Regression
- If user selects specific models, adds only those; otherwise defaults to the full priority list
- Attaches a param_grid dict to each tunable model (RF, XGB, GBM)
- Non-tunable models (LogReg, SVM, Ridge) receive an empty grid and skip GridSearchCV

Outputs:

- model_list: list of (name, estimator, param_grid) tuples

UI Display:

- Model selection checkboxes shown in Phase 2 Configuration panel
- Selected models displayed as badges before pipeline execution

5.6 Agent 6: Training Agent

Purpose: Trains each selected model, runs hyperparameter search where applicable, and triggers a retry loop if the best F1 falls below the minimum threshold.

Inputs:

- X_train, y_train
- model_list from Model Selector

Processing Logic:

- Iterates over model_list; for models with non-empty param_grid, runs GridSearchCV(cv=3, scoring='f1_weighted')
- Stores best_estimator_ and best_score_ for each model
- After all models trained, checks if max(scores) < 0.65; if so, conditional edge routes back to this agent with increased n_estimators and reduced learning_rate
- Retry limit capped at 2 to prevent infinite loops

Outputs:

- trained_models dict: {model_name: {estimator, score, train_time, tuned_flag}}

UI Display:

- Live agent status badge updates to "Training..." during execution
- Each model result appears in Model Comparison tab with TUNED badge

5.7 Agent 7: Evaluation Agent

Purpose: Selects the best model and produces a comprehensive evaluation report including confusion matrix and per-class metrics.

Inputs:

- trained_models
- X_test, y_test

Processing Logic:

- Ranks models by f1_weighted on test set; sets best_model_name
- Generates classification_report(output_dict=True) for per-class precision, recall, F1
- Computes confusion_matrix and converts to nested list for JSON serialisation
- Stores macro F1, weighted F1, and per-class F1 separately to avoid misleading aggregation on imbalanced datasets

Outputs:

- evaluation_report dict
- confusion_matrix list[list]
- best_model_name str

UI Display:

- Confusion matrix rendered as interactive colour-coded grid
- Per-class F1 bar chart with class labels
- Best model highlighted with gold trophy icon

5.8 Agent 8: Explainability Agent

Purpose: Computes feature importances using SHAP for tree-based models and coefficient magnitudes for linear models.

Inputs:

- trained_models[best_model_name]
- X_test
- feature_names

Processing Logic:

- For XGBoost and Random Forest: uses shap.TreeExplainer(model).shap_values(X_test)
- Aggregates SHAP values by mean absolute value per feature; sorts descending
- For Logistic Regression / Ridge: uses abs(model.coef_[0]) directly
- Normalises importances to 0–1 scale for frontend rendering
- Handles multi-class SHAP by averaging across classes

Outputs:

- feature_importances dict: {feature_name: importance_score}

UI Display:

- Horizontal bar chart sorted by importance
- Top-10 features highlighted; remainder collapsible
- Colour-coded bars: blue for SHAP-derived, teal for coefficient-derived

5.9 Agent 9: Report Agent

Purpose: Synthesises all agent outputs into a structured natural-language final report available for download.

Inputs:

- Full state dict (all keys)

Processing Logic:

- Constructs a structured Markdown string covering: dataset summary, cleaning actions, EDA highlights, preprocessing decisions, model comparison, best model metrics, and top-5 feature importances
- Optionally calls Groq LLM API (llama-3.3-70b-versatile) to generate narrative paragraphs for each section when LLM_MODE is enabled
- Stores final_report string in state for frontend rendering and download

Outputs:

- final_report: formatted string (Markdown or LLM-narrated)

UI Display:

- Report tab renders formatted Markdown with syntax highlighting
- Download button triggers browser file download of report.md

KEY TAKEAWAY *The nine-agent pipeline turns a 200+ line monolithic notebook into a modular, auditable graph where each node has a single responsibility, making debugging, testing, and future extension dramatically easier.*

6. Frontend Design

6.1 Application Flow — 3-Phase Wizard

Phase 1 — File Upload

- Drag-and-drop upload zone accepts CSV, Excel, and Parquet files up to 50 MB.
- On drop, Axios POST /upload streams the file to FastAPI; a preview table renders the first 10 rows.
- Column list with dtypes displayed; user confirms before proceeding.

Phase 2 — Configuration

- Target column selector: dropdown populated from uploaded file columns.
- Goal text field: free-text description of the analysis objective.
- Model selection: checkboxes for each available algorithm; "Auto-Select All" toggle.
- Mode toggle: Full Mode (SHAP + all visualisations) vs Fast Mode (skip heavy computations).

Phase 3 — Results Dashboard

- Triggered on "Run Analysis" click; POST /analyze returns job_id.
- Polling loop: GET /status/{job_id} every 2 seconds updates live agent status badges.
- On completion, seven result tabs load with all agent outputs.

6.2 Results Dashboard Tabs

Tab	Content	Key Visualisations
Data Cleaning	Quality score, cleaning log	Animated gauge, action audit list, stats cards
EDA	Statistical summaries, distributions	Correlation heatmap, box plots, donut chart
Preprocessing	Feature engineering decisions	Scaler used, SMOTE flag, train/test sizes
Model Comparison	All models ranked by F1	Sortable table with F1, train time, TUNED badge
Evaluation	Best model deep dive	Confusion matrix grid, per-class F1 bar chart
Explainability	Feature importance scores	Horizontal bar chart (SHAP / coef_)
Report	Final NL summary	Formatted Markdown + download button

6.3 State Management

- No Redux or Zustand — single-session data managed via React useState hooks; data does not persist across page refreshes by design.
- Axios instance configured with baseUrl=http://localhost:8000 and interceptors for loading and error state management.
- Job queue pattern: POST /analyze returns job_id immediately; frontend polls GET /status/{job_id} every 2 seconds until status === "complete" or "error".
- This pattern prevents frontend HTTP timeout on 10–30 second Full Mode pipeline runs.
- All result data stored in a single results state object, destructured into each tab component via props.

KEY TAKEAWAY *The polling architecture is essential for UX on long-running pipelines: it decouples the HTTP request lifecycle from the LangGraph execution time, preventing browser timeout errors and enabling live progress feedback.*

7. Testing & Results

7.1 Test Datasets Overview

Dataset	Rows	Features	Task	Target	Imbalance Ratio
Iris	150	4	Multi-class Classification	Species	1.0 (balanced)
Heart Disease	303	13	Binary Classification	target	~1.1
Telco Customer Churn	7,043	20	Binary Classification	Churn	2.77
Insurance Charges	1,338	6	Regression	charges	N/A
House Prices	1,460	79	Regression	SalePrice	N/A

7.2 Telco Customer Churn — Detailed Case Study

Dataset Characteristics:

- 7,043 rows; 20 feature columns; binary target: Churn (Yes/No).
- Class imbalance: 73.5 % No-Churn vs 26.5 % Churn → imbalance_ratio = 2.77 → SMOTE triggered.
- Notable ID column: customerID (7,043 unique values) — automatically detected and dropped by Cleaning Agent.
- Mixed types: 16 categorical + 4 numeric; TotalCharges stored as string with whitespace — imputed by Cleaning Agent.

Rank	Model	F1 Score	Train Time	Tuned?
1	XGBClassifier	0.7973	1.8 s	Yes
2	GradientBoosting	0.7842	5.7 s	Yes
3	RandomForest	0.7811	1.5 s	Yes
4	LogisticRegression	0.7488	1.1 s	No

XGBoost Best Hyperparameters:

```
{subsample: 0.7, n_estimators: 300, max_depth: 3, learning_rate: 0.05}
```

Confusion Matrix — XGBClassifier on Telco Churn:

	Predicted: No	Predicted: Yes
Actual: No	939	96
Actual: Yes	178	196

Churn Recall = 52.4 % (imbalance issue)
 Churn Precision = 67.1 %
 Weighted F1 = 0.7973

7.3 Bugs Identified and Fixed

#	Bug Description	Root Cause	Fix Applied
1	customerID encoded as 7043-class feature	No ID-column detection	Auto-drop cols where unique_ratio > 0.95
2	SMOTE not applied despite 2.77x imbalance	Threshold not set	Auto-SMOTE when imbalance_ratio > 1.5
3	Negative Y-axis on missing values chart	No axis bounds set	Explicit ylim on chart render
4	Tooltip showing "value: 0" on LogReg bar	Wrong data key in Recharts component	Corrected key mapping in frontend component
5	Weighted F1 misleading for imbalanced data	Single metric shown	Added per-class F1 and macro F1 to display

KEY TAKEAWAY *The customerID bug is a canonical example of why automated ID-column detection is critical: encoding a 7,043-cardinality column as features would have made the model memorise training indices rather than learn generalizable patterns.*

8. Challenges & Solutions

Challenge 1 — LangGraph State Management (Agents Returning None)

Problem:

- Initial agents returned partial state updates, causing downstream agents to receive None for expected keys.
- LangGraph requires agent functions to return the full updated state dict, not just modified keys.

Solution:

- Refactored all agent functions to explicitly merge their outputs with the incoming state: return `{**state, 'new_key': value}`.
- Added state validation middleware that checks for required keys before each node executes.

Impact: Eliminated all NoneType errors and made pipeline execution fully deterministic.

Challenge 2 — Task Type Inference Ambiguity (int64 Target)

Problem:

- Binary-encoded targets (0/1 integer) were misclassified as REGRESSION by the EDA Agent.
- Cardinality threshold alone is insufficient — a column of 0s and 1s has `nunique()=2` but represents classification.

Solution:

- Extended inference logic: if `nunique() <= 2` and `dtype` is numeric → always CLASSIFICATION.
- Added a manual override field in the Phase 2 configuration panel for edge cases.

Impact: Correct task type inference on all five test datasets with zero misclassifications.

Challenge 3 — Frontend Timeout on Long Pipeline Runs

Problem:

- Direct await on POST /analyze caused browser timeout for Full Mode runs exceeding 30 seconds.
- React's default Axios timeout of 10 seconds killed the connection mid-pipeline.

Solution:

- Implemented job queue pattern: POST /analyze returns job_id immediately; frontend polls GET /status/{job_id} every 2 seconds.
- Increased Axios timeout to 60 seconds as a safety net.

Impact: Zero timeout errors in all test runs; live per-agent progress visible to the user.

Challenge 4 — CORS Errors in Development

Problem:

- React dev server at localhost:5173 was blocked by FastAPI's default CORS policy.
- Preflight OPTIONS requests failed, preventing all API calls.

Solution:

- Added CORSMiddleware with allow_origins=["http://localhost:5173"], allow_methods=["*"], allow_headers=["*"].
- Documented the required environment variable BACKEND_URL for production deployment.

Impact: Seamless React ↔ FastAPI communication in all environments.

Challenge 5 — Data Leakage in Feature Engineering

Problem:

- Early implementation fitted StandardScaler on the full dataset before splitting, causing test-set information to leak into training.
- One-hot encoder fitted on full data produced unseen-category issues on small datasets.

Solution:

- Moved all transformer fitting inside the train split; applied fitted transformers to test split separately.
- Wrapped scaler and encoder inside scikit-learn Pipeline objects to enforce correct fit/transform sequencing.

Impact: Elimination of data leakage; evaluation metrics now reflect true generalisation performance.

KEY TAKEAWAY The data leakage challenge highlights a systemic risk in ad-hoc notebook-based pipelines — InsightForge's agent architecture enforces correct sequencing by design, making leakage structurally impossible once the fix was applied.

9. Future Scope

Phase	Feature	Benefit
Short-term	Complete SMOTE auto-application	Better recall on imbalanced datasets
Short-term	Per-class F1 display	Clearer evaluation for skewed targets
Short-term	Server-Sent Events (SSE) for real-time progress	Live per-agent streaming in UI
Medium-term	LLM in report_agent for richer NL insights	Natural language narratives per finding
Medium-term	Model persistence + POST /predict endpoint	Real-time inference on new rows
Medium-term	Time-series pipeline (ARIMA / Prophet)	Sequential data support
Medium-term	PDF report export	Professional deliverable format
Long-term	Optuna AutoML replacing GridSearchCV	Smarter hyperparameter search
Long-term	Docker + GitHub Actions CI/CD	Production-ready deployment
Long-term	Cloud source integrations (BigQuery, S3)	Enterprise data connectivity
Long-term	NLP / text classification pipeline	Unstructured data support

KEY TAKEAWAY *The roadmap is intentionally sequenced: short-term fixes address known evaluation gaps; medium-term features unlock real production use cases; long-term investments transform InsightForge from a prototype into an enterprise-grade platform.*

10. Conclusion

- InsightForge successfully automates the complete data science pipeline — from raw CSV upload to final model report — eliminating the repetitive boilerplate that consumes the majority of a data scientist's project time.
- The nine-agent LangGraph architecture provides clear separation of concerns, auditable state transitions, and a conditional retry mechanism that ensures minimum model quality before proceeding to evaluation.
- On the Telco Customer Churn benchmark, InsightForge autonomously selected, tuned, and evaluated XGBoost to a weighted F1 score of 0.7973 — competitive with manually engineered baselines — in under 2 seconds of training time.
- Key engineering challenges — data leakage prevention, task type inference, frontend timeout handling, and state management — were resolved through principled architectural decisions rather than workarounds.

- The platform democratizes machine learning for non-technical domain experts: a clinician, analyst, or product manager can upload a CSV, describe their goal in plain language, and receive a professional ML evaluation report in under 60 seconds.
- InsightForge demonstrates that the multi-agent agentic paradigm is not merely a research curiosity but a practical engineering pattern that reduces code complexity, improves reproducibility, and accelerates iteration.

KEY TAKEAWAY *As LLM-orchestrated agentic systems mature, platforms like InsightForge will become the standard interface between domain expertise and machine learning capability — making AI not just accessible but truly useful for every knowledge worker.*

11. References

1. Ramírez, S. (2024). FastAPI: Modern, Fast Web Framework for Building APIs with Python. tiangolo.com/fastapi.
2. LangChain Inc. (2024). LangGraph: Building Stateful Multi-Agent Applications. <https://github.com/langchain-ai/langgraph>.
3. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
4. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
5. Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30 (NeurIPS 2017).
6. Meta Platforms Inc. (2024). React: The Library for Web and Native User Interfaces. <https://react.dev>.
7. You, E. (2024). Vite: Next Generation Frontend Tooling. <https://vitejs.dev>.
8. Groq Inc. (2024). Groq API Documentation. <https://console.groq.com/docs>.